Visvesvaraya Technological University, Belagavi

Department of AI & ML Engineering

AKASH INSTITUTE OF ENGINEERING AND TECHNOLOGY

1BCS302 - Java Lab Manual

for

III Semester BE (AI&ML, CY)

Lab Incharge: Mr. Venkatesh K

Vision

To establish an inclusive and excellence-driven learning environment in Artificial Intelligence and Machine Learning, providing students with quality technical and practical education, industry-relevant skills, and opportunities for innovation, while empowering them to contribute meaningfully to society and the technological development of rural and underserved communities.

Mission

1. To provide quality education in Artificial Intelligence and Machine Learning by developing students' problem-solving, analytical, computational, and practical skills for successful careers and higher studies.
2. To promote self-learning, research, innovation, and entrepreneurship by encouraging students to explore emerging technologies and develop solutions to real-world problems, particularly those relevant to rural and underserved communities.
3. To impart value-based and industry-oriented education through modern tools, hands-on learning, projects, and exposure to current advancements in AI, ML, data science, and high-performance computing.
4. To bridge the digital and technological gap by creating opportunities for students from diverse and rural backgrounds to acquire relevant technical competencies and participate confidently in the evolving digital economy.
5. To encourage socially responsible applications of AI and ML and contribute to the development of intelligent, affordable, and sustainable solutions addressing local and societal needs.

Programme Educational Objectives

PSO1

Students will be able to apply computer programming approach to deliver solutions for real-world challenges with emerging techniques.

PSO2

Students will be able to analyze, implement, and deploy suitable AI and ML models across various domains.

Program Outcome

PO1: Engineering Knowledge

Apply the knowledge of mathematics, science, engineering fundamentals, and computer science and engineering to the solution of complex engineering problems.

PO2: Problem Analysis

Identify, formulate, review research literature, and analyse complex computer engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO3: Design/Development of Solutions

Design solutions for complex real-world problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO4: Conduct Investigations of Complex Problems

Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO5: Modern Tool Usage

Create, select, and apply appropriate techniques, resources, and modern Machine Learning and AI tools including prediction and modelling to complex computer engineering activities with an understanding of the limitations.

PO6: The Engineer and Society

Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO7: Environment and Sustainability

Understand the impact of the professional AI & ML solutions in societal and environmental contexts, and demonstrate the

knowledge of, and need for sustainable development.

PO8: Ethics

Apply ethical principles and commit to professional ethics and responsibilities and norms of the AI & ML engineering practice.

PO9: Individual and Team Work

Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

PO10: Communication

Communicate effectively on complex AI & ML activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO11: Project Management and Finance

Demonstrate knowledge and understanding of the AI & ML engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO12: Life-long Learning

Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

Course Name	JAVA Lab		
Course Code	1BCS302	CIE Marks	50
Number of Lecture Hours/Week	0.5 Hr Tutorial (Instructions) + 1.5 Hours Laboratory	SEE Marks	50

CO	COURSE OUTCOMES	RBT
CO1	Demonstrate proficiency in writing simple programs involving branching and looping structures.	L1, L2, L3
CO2	Construct object-oriented solutions using classes, objects, and methods.	
CO3	Develop programs using inheritance, packages, and interfaces.	
CO4	Make use of exception handling and multithreading in solving complex problems	
CO5	Apply concepts of autoboxing, enumerations, and generics in program development.	
CO6	Choose a suitable IDE tool to design, develop, and debug Java programs.	

COURSE OBJECTIVE

CONTENTS

1. Develop a Java program to add TWO matrices of suitable order N.
2. Develop a stack class to hold a maximum of 10 integers with suitable methods. Develop a Java main method to illustrate Stack operations.
3. Create a Java class called Student with the USN,Name,Branch,Phone details as variables within it. Write a Java program to create N Student objects and print the USN, Name, Branch, and Phone of these objects with suitable headings.
4. Develop a Java program to create a class named Shape. Create three sub classes, namely: circle, triangle and square. Each class has two member functions named draw and erase. Demonstrate the concepts of polymorphism by developing appropriate methods, defining member data, and implementing the main program.
5. Develop a Java program to create an interface Resizable with methods resizeWidth(int width) and resizeHeight(int height) that allow an object to be resized. Create a class Rectangle that implements the Resizable

interface and implements the resize methods.
6. Develop a Java program to create a package named mypack and import & implement it in a suitable class.
7. Develop a Java program to raise a custom exception (user-defined exception) for DivisionByZero using try, catch, throw and finally.
8. Develop a program to create a class MyThread with a class constructor, call the base class constructor, using super and start the thread. The run method of the class starts after this. It can be observed that both main thread and created child thread are executed concurrently.
9. Develop a Java program using an enumeration called Department containing CSE, ISE, ECE, and MECH. Accept a department name as input and display the selected department using valueOf().
10. Develop a Java program to read the marks of three subjects as strings, convert them into wrapper objects, calculate the total and average using autoboxing/unboxing and display the result.
11. **Micro Project:** A team of two to three students shall develop a project and submit a brief report for a specific task assigned by the course instructor based on the concepts covered in the syllabus.

EXPERIMENT: 1

AIM: To develop a Java program to add TWO matrices of suitable order N

THEORY:

Matrix Addition is the operation of adding two matrices by adding the corresponding elements together.

- **Dimensional Requirement:** Both matrices must have the exact same dimensions ($N \times N$).
- **Element-wise Addition:** For any element at row i and column j:

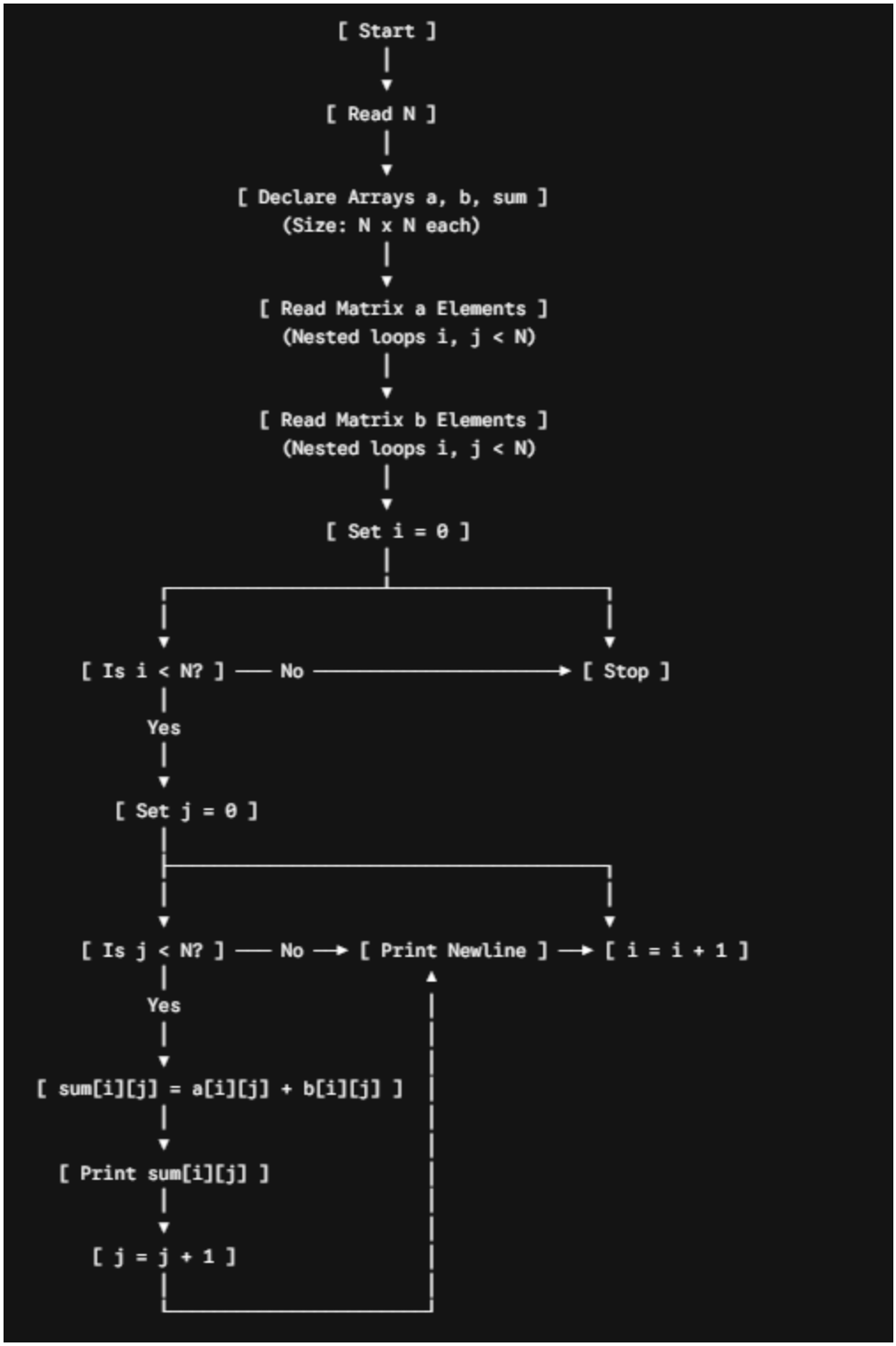
$$\text{sum}[i][j] = a[i][j] + b[i][j]$$

- **Time Complexity:** $O(N^2)$ because we use nested loops to iterate through all $N \times N$ elements of the matrices.

ALGORITHM

1. **Start**
2. Read the matrix order N from the user.
3. Declare three 2D integer arrays of size $N \times N$: a, b, and sum.
4. **Input First Matrix (a):**
    - Loop i from 0 to N - 1:
        - Loop j from 0 to N - 1:
            - Read a[i][j].
5. **Input Second Matrix (b):**
    - Loop i from 0 to N - 1:
        - Loop j from 0 to N - 1:
            - Read b[i][j].
6. **Compute and Display Sum:**
    - Loop i from 0 to N - 1:
        - Loop j from 0 to N - 1:
            - Compute sum[i][j] = a[i][j] + b[i][j].
            - Print sum[i][j] followed by a space.
        - Print a newline after finishing each row i.
7. **Stop**

FLOWCHART



PROGRAM:

```
import java.util.Scanner;

class MatrixAddition {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

      System.out.print("Enter order N: ");
      int n = sc.nextInt();
```

```
        int[][] a = new int[n][n];
        int[][] b = new int[n][n];
        int[][] sum = new int[n][n];

        System.out.println("Enter first matrix:");
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                a[i][j] = sc.nextInt();

        System.out.println("Enter second matrix:");
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                b[i][j] = sc.nextInt();

        System.out.println("Sum of matrices:");
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                sum[i][j] = a[i][j] + b[i][j];
                System.out.print(sum[i][j] + " ");
            }
            System.out.println();
        }
    }
}
```

RESULTS:

EXPERIMENT: 2

AIM:

Develop a stack class to hold a maximum of 10 integers with suitable methods. Develop a Java main method to illustrate Stack operations.

Theory

A Stack is a linear data structure that follows the LIFO (Last-In, First-Out) principle. This means the element inserted last is the first one to be removed.

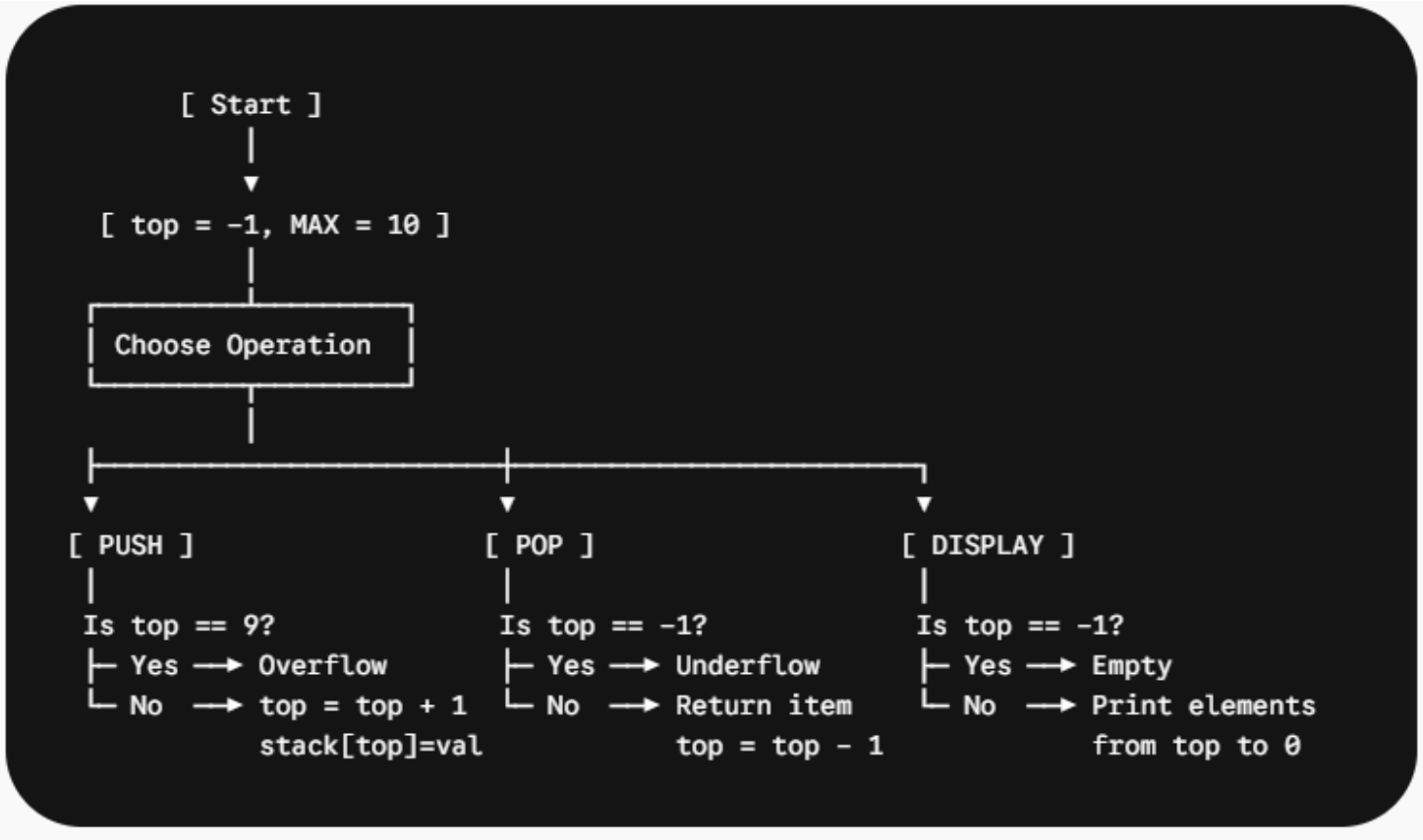
Key Operations:

- Push: Adds an integer to the top of the stack (triggers Overflow if the stack is full).
- Pop: Removes and returns the top integer (triggers Underflow if the stack is empty).
- Peek / Display: Views the top element or prints the stack content without modifying it.

Algorithm

1. ***Initialize:*** *Define an array stack of size 10 and a variable top = -1.*
2. ***Push(val):***
    - *If top == 9, display "**Stack Overflow**".*
    - *Else, increment top (top = top + 1) and set stack[top] = val.*
3. ***Pop():***
    - *If top == -1, display "**Stack Underflow**".*
    - *Else, return stack[top] and decrement top (top = top - 1).*
4. ***Display():***
    - *If top == -1, print "**Stack is empty**".*
    - *Else, loop from top down to 0 and print each element.*

Flowchart Structure



Java Program

Java

```
class Stack {
```

```
    private final int MAX = 10;
    private int[] stack = new int[MAX];
    private int top = -1;

    // Push operation
    public void push(int val) {
        if (top == MAX - 1) {
            System.out.println("Stack Overflow! Cannot push " + val);
        } else {
            stack[++top] = val;
            System.out.println("Pushed: " + val);
        }
    }

    // Pop operation
    public int pop() {
        if (top == -1) {
            System.out.println("Stack Underflow!");
            return -1;
        } else {
            return stack[top--];
        }
    }

    // Display stack contents
    public void display() {
        if (top == -1) {
            System.out.println("Stack is empty.");
            return;
        }
        System.out.print("Stack (top to bottom): ");
        for (int i = top; i >= 0; i--) {
            System.out.print(stack[i] + " ");
        }
        System.out.println();
    }
}

public class Main {
    public static void main(String[] args) {
        Stack s = new Stack();

        // Demonstrating stack operations
        s.push(10);
        s.push(20);
        s.push(30);
        s.display();

        System.out.println("Popped element: " + s.pop());
```

```
        s.display();
    }
}
```

RESULT:

EXPERIMENT: 3

AIM: Create a Java class called Student with the USN,Name,Branch,Phone details as variables within it. Write a Java program to create N Student objects and print the USN, Name, Branch, and Phone of these objects with suitable headings.

THEORY

An Array of Objects in Java allows storing multiple instances of a class in a single continuous block of memory.

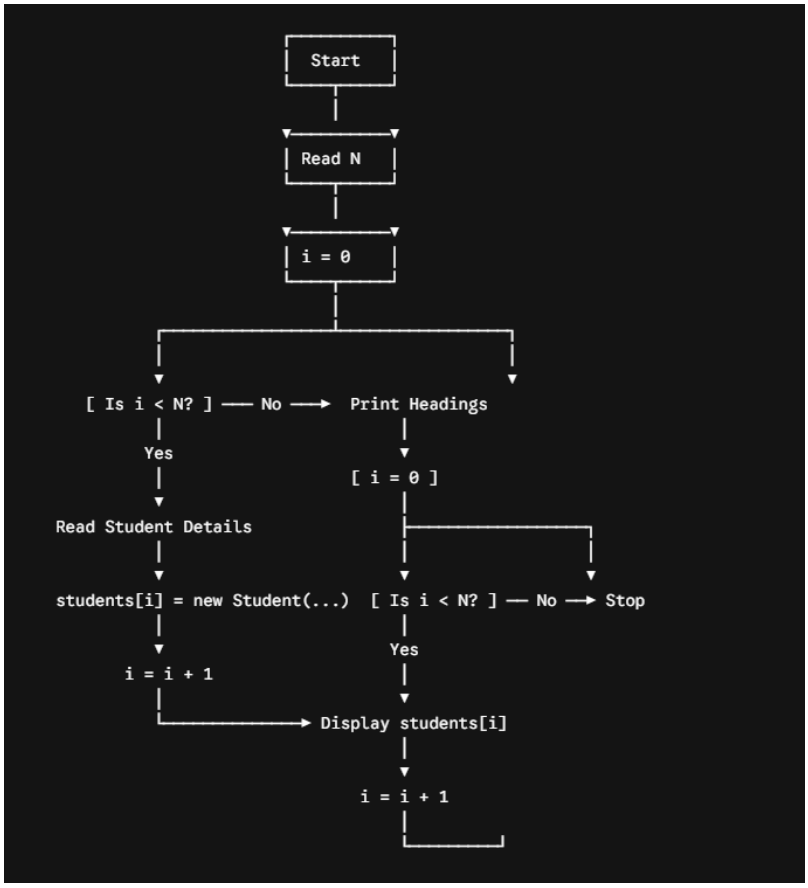
- Object Encapsulation: The Student class acts as a blueprint grouping distinct attributes (USN, Name, Branch, Phone) into a single logical unit.
- Dynamic Allocation: By taking N as input at runtime, we dynamically create an array of references:

$$\text{Student[] students} = \text{new Student}[N]$$

- Memory Step: Instantiating the array creates references (N null pointers). Each element must then be individually instantiated using new Student(...).

ALGORITHM

1. ***Start***
2. *Read the total number of students, N.*
3. *Declare an array students of size N.*
4. *Set loop counter* $i = 0$.
5. ***While*** $i < N$***:***
    - *Read USN, Name, Branch, and Phone.*
    - *Create a new Student object with these values.*
    - *Store the object at students[i].*
    - *Increment* $i = i + 1$.
6. *Print tabular headers: USN, NAME, BRANCH, PHONE.*
7. *Set loop counter* $i = 0$.
8. ***While*** $i < N$***:***
    - *Call display() on students[i] to output attributes in formatted columns.*
    - *Increment* $i = i + 1$.
9. ***Stop***




```
import java.util.Scanner;

class Student {
    // Instance variables
    String usn;
    String name;
    String branch;
    long phone;

    // Constructor to initialize student details
    public Student(String usn, String name, String branch, long phone) {
        this.usn = usn;
        this.name = name;
        this.branch = branch;
        this.phone = phone;
    }

    // Method to display student details in tabular format
    public void display() {
        System.out.printf("%-15s %-20s %-15s %-15d\n", usn, name, branch, phone);
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter the number of students (N): ");
        int n = scanner.nextInt();
        scanner.nextLine(); // Consume newline
```

```
        // Array to store N Student objects
        Student[] students = new Student[n];

        // Input student details
        for (int i = 0; i < n; i++) {
            System.out.println("\nEnter details for Student " + (i + 1) + ":");

            System.out.print("USN: ");
            String usn = scanner.nextLine();

            System.out.print("Name: ");
            String name = scanner.nextLine();

            System.out.print("Branch: ");
            String branch = scanner.nextLine();

            System.out.print("Phone:  ");
            long phone = scanner.nextLong();
            scanner.nextLine(); // Consume newline

            // Creating a new Student object and storing it in the array
            students[i] = new Student(usn, name, branch, phone);
        }

        // Displaying student details with suitable headings
        System.out.println("\n======================================================================");
        System.out.printf("%-15s %-20s %-15s %-15s\n", "USN", "NAME", "BRANCH", "PHONE");
        System.out.println("======================================================================");

        for (int i = 0; i < n; i++) {
            students[i].display();
        }

        System.out.println("======================================================================");

        scanner.close();
    }
}
```

RESULT

EXPERIMENT: 4

AIM: Develop a Java program to create a class named Shape. Create three sub classes, namely: circle, triangle and square. Each class has two member functions named draw and erase. Demonstrate the concepts of polymorphism by developing appropriate methods, defining member data, and implementing the main program.

THEORY

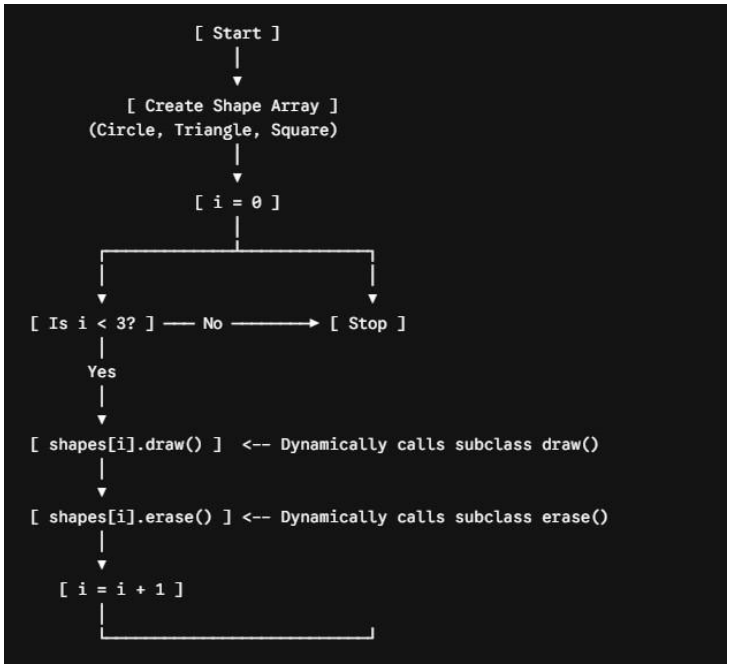
Polymorphism (meaning "many forms") allows us to perform a single action in different ways. In Java, this is achieved using **method overriding** (runtime polymorphism):

- **Superclass Reference, Subclass Object:** A superclass reference variable (Shape s) can hold an object of any of its subclasses (Circle, Triangle, Square).
- **Dynamic Method Dispatch:** When overridden methods like draw() or erase() are called, Java determines *at runtime* which class's implementation to execute based on the actual object being referred to.

ALGORITHM

1. **Start**
2. Define a base class Shape with virtual/generic methods draw() and erase().
3. Create subclasses Circle, Triangle, and Square extending Shape.
4. Override draw() and erase() in each subclass with specific print messages.
5. In main():
    - Create an array of Shape references holding instances of Circle, Triangle, and Square.
6. Loop through each Shape object in the array and call draw() and erase().
7. **Stop**

FLOWCHART



PROGRAM

```
// Base class
class Shape {
    void draw() {
        System.out.println("Drawing a generic shape");
    }

    void erase() {
        System.out.println("Erasing a generic shape");
    }
}
```

```
// Subclass 1
class Circle extends Shape {
    @Override
    void draw() {
        System.out.println("Drawing a Circle");
    }

    @Override
    void erase() {
        System.out.println("Erasing a Circle");
    }
}

// Subclass 2
class Triangle extends Shape {
    @Override
    void draw() {
        System.out.println("Drawing a Triangle");
    }

    @Override
    void erase() {
        System.out.println("Erasing a Triangle");
    }
}

// Subclass 3
class Square extends Shape {
    @Override
    void draw() {
        System.out.println("Drawing a Square");
    }

    @Override
    void erase() {
        System.out.println("Erasing a Square");
    }
}

// Main class to demonstrate Polymorphism
public class Main {
    public static void main(String[] args) {
        // Superclass reference array holding subclass objects
        Shape[] shapes = new Shape[3];
        shapes[0] = new Circle();
        shapes[1] = new Triangle();
        shapes[2] = new Square();

        // Demonstrating Runtime Polymorphism
        for (Shape shape : shapes) {
            shape.draw();
            shape.erase();
            System.out.println(" --------------------");
        }
    }
}
```

RESULT

EXPERIMENT: 5

AIM

Develop a Java program to create an interface Resizable with methods resizeWidth(int width) and resizeHeight(int height) that allow an object to be resized. Create a class Rectangle that implements the Resizable interface and implements the resize methods.

THEORY

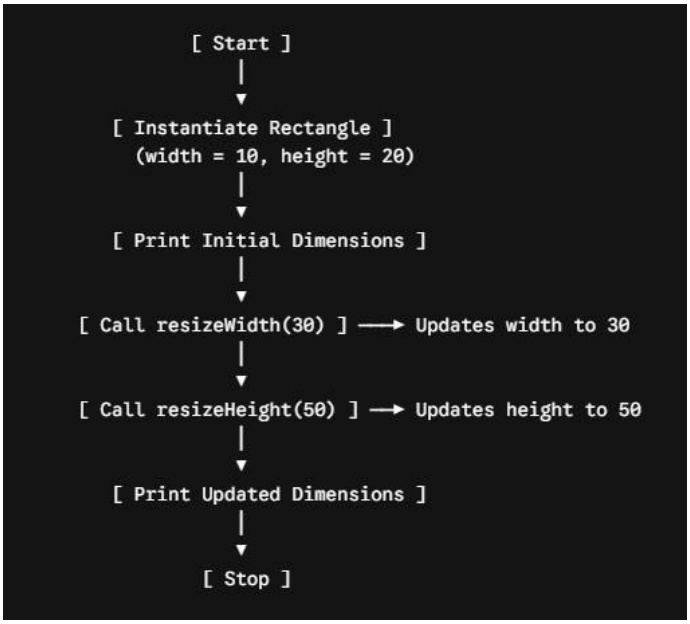
An **Interface** in Java is a blueprint of a class containing abstract methods (methods without a body) that subclasses must implement.

- **Interface Contract:** The Resizable interface forces any implementing class to provide concrete code for resizeWidth(int width) and resizeHeight(int height).
- **Abstraction:** Interfaces separate *what* an object should do (resize) from *how* it actually does it (modifying its specific fields).

ALGORITHM

1. **Start**
2. Define interface Resizable with method signatures resizeWidth(int) and resizeHeight(int).
3. Define class Rectangle that implements Resizable with width and height attributes.
4. Implement resizeWidth(int newWidth) to update the width attribute.
5. Implement resizeHeight(int newHeight) to update the height attribute.
6. In main():
    - Instantiate a Rectangle object with initial dimensions.
    - Display initial width and height.
    - Call resizeWidth() and resizeHeight() with new values.
    - Display updated dimensions.
7. Stop

FLOWCHART



PROGRAM

```
// Interface definition
interface Resizable {
    void resizeWidth(int width);
    void resizeHeight(int height);
}

// Rectangle class implementing Resizable
```

```
class Rectangle implements Resizable {
    private int width;
    private int height;

    // Constructor
    public Rectangle(int width, int height) {
        this.width = width;
        this.height = height;
    }

    // Implementing interface methods
    @Override
    public void resizeWidth(int width) {
        this.width = width;
        System.out.println("Width resized to: " + this.width);
    }

    @Override
    public void resizeHeight(int height) {
        this.height = height;
        System.out.println("Height resized to: " + this.height);
    }

    // Method to display current dimensions
    public void display() {
        System.out.println("Rectangle Dimensions: " + width + " x " + height);
    }
}

// Main class
public class Main {
    public static void main(String[] args) {
        // Creating a Rectangle object
        Rectangle rect = new Rectangle(10, 20);

        System.out.println("--- Initial State ---");
        rect.display();

        System.out.println("\n--- Resizing ---");
        rect.resizeWidth(35);
        rect.resizeHeight(50);

        System.out.println("\n--- Final State ---");
        rect.display();
    }
}
```

RESULT

EXPERIMENT: 6

AIM
Develop a Java program to create a package named mypack and import & implement it in a suitable class.

THEORY
A **Package** in Java is a mechanism to group related classes, interfaces, and sub-packages together.

- **Namespace Management:** Packages prevent naming conflicts (e.g., two classes named Calculator can exist in different packages).
- **Access Control:** Code inside a package can restrict access using default or protected modifiers.
- **Import Keyword:** The import keyword allows classes in one package to access public classes in another package.

ALGORITHM

1. Start
2. Create a folder named mypack.
3. Inside mypack, define a public class Message with a package declaration package mypack;.
4. Define a public method display() in Message that prints a statement.
5. Compile Message.java.
6. Outside mypack, create a main class TestPackage and write import mypack.Message;.
7. In main(), instantiate Message and invoke display().
8. **Stop**

FLOWCHART



JAVA PROGRAM

File 1: mypack/Message.java
(Save this file inside a folder named mypack)
Java

```
package mypack;

public class Message {
    public void display() {
```

```
        System.out.println("Hello from mypack package!");
    }
}
```

File 2: TestPackage.java

(Save this file outside the mypack folder)

Java

```
import mypack.Message;

public class TestPackage {
    public static void main(String[] args) {
        // Creating an object of class from 'mypack' package
        Message msg = new Message();

        // Calling method from imported class
        msg.display();
    }
}
```

How to Run (Command Line)

Bash

```
# 1. Compile the package class
javac mypack/Message.java

# 2. Compile and run the main class
javac TestPackage.java
java TestPackage
```

RESULT

EXPERIMENT: 7

AIM

Develop a Java program to raise a custom exception (user-defined exception) for DivisionByZero using try, catch, throw and finally

THEORY

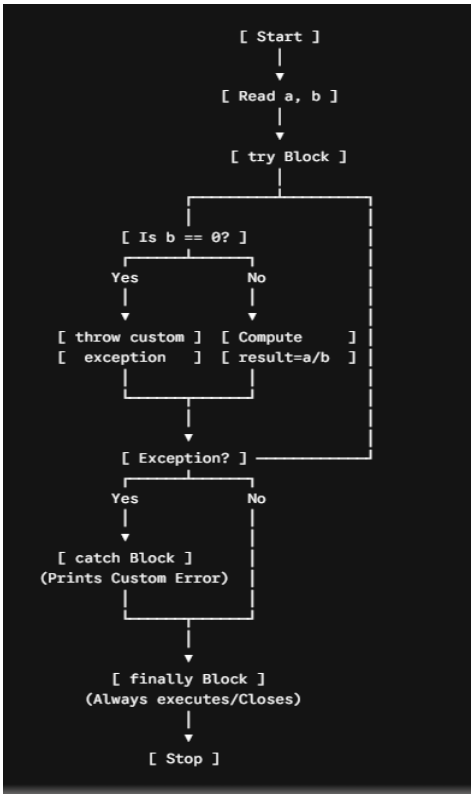
An **Exception** is an unwanted or unexpected event that disrupts the normal flow of a program during execution.

- **Custom (User-Defined) Exception:** Created by extending Java's built-in Exception class to handle application-specific error conditions.
- **Keywords Used:**
  - **try:** Encloses code that might throw an exception.
  - **throw:** Explicitly triggers a custom exception when division by zero occurs ($b = 0$).
  - **catch:** Intercepts and handles the thrown exception safely without crashing the program.
  - **finally:** Executes code *always* (e.g., closing resources), regardless of whether an exception occurred or was caught.

ALGORITHM

1. **Start**
2. Define a custom exception class DivisionByZeroException extending Exception.
3. Read numerator a and denominator b from the user.
4. Enter try block:
   - If $b == 0$, **throw** DivisionByZeroException("Cannot divide by zero!").
   - Else, calculate $result = a/b$ and print it.
5. Enter catch block (executes only if $b == 0$):
   - Print the custom error message.
6. Enter finally block:
   - Print a completion message and close the scanner resource.
7. Stop

FLOWCHART



PROGRAM

```
import java.util.Scanner;

// Step 1: Define Custom Exception
class DivisionByZeroException extends Exception {
    public DivisionByZeroException(String message) {
        super(message);
    }
}

public class Main {
    // Method that throws the custom exception
    public static double divide(int a, int b) throws DivisionByZeroException {
        if (b == 0) {
            // Explicitly throw user-defined exception
            throw new DivisionByZeroException("Error: Custom Exception - Division by zero is not
allowed!");
        }
        return (double) a / b;
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        try {
            System.out.print("Enter numerator (a): ");
            int a = scanner.nextInt();

            System.out.print("Enter denominator (b): ");
            int b = scanner.nextInt();

            // Calling divide method
            double result = divide(a, b);
            System.out.println("Result: " + result);

        } catch (DivisionByZeroException e) {
            // Catching custom exception
            System.out.println(e.getMessage());
        } finally {
            // Always executes regardless of exception
            System.out.println("\n[finally block] Cleanup completed. Program execution ended.");
            scanner.close();
        }
    }
}
```

EXPERIMENT: 8

AIM

Develop a program to create a class MyThread with a class constructor, call the base class constructor, using super and start the thread. The run method of the class starts after this. It can be observed that both main thread and created child thread are executed concurrently.

THEORY

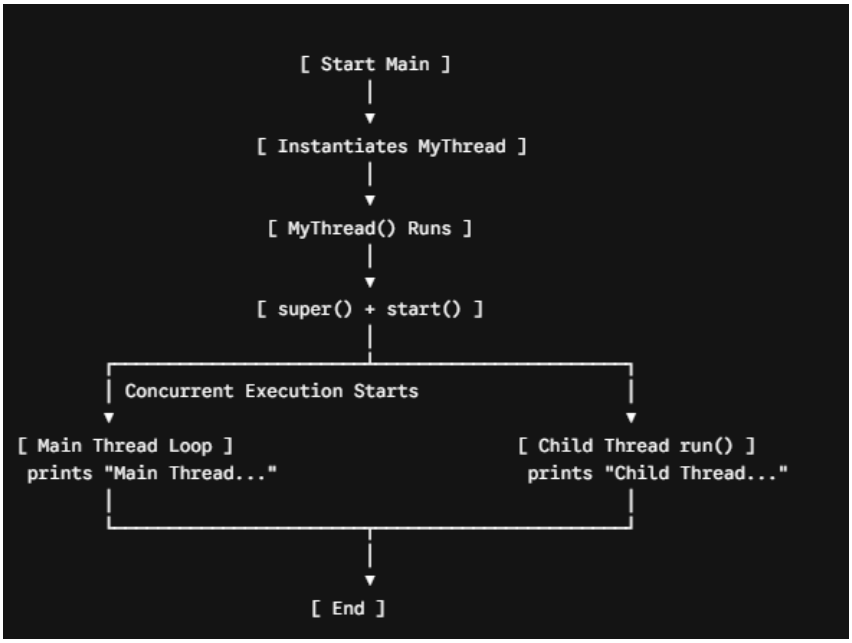
Multithreading allows Java programs to execute multiple tasks simultaneously to maximize CPU usage.

- **Extending Thread Class:** A custom class creates a thread by inheriting from java.lang.Thread.
- **The super() Call:** Calling super(threadName) inside the constructor passes the custom name to the base Thread class constructor.
- **Concurrency:** Calling .start() allocates a new call stack and executes the run() method asynchronously. While the child thread loops, the main thread continues executing independently on its own thread stack.

ALGORITHM

1. **Start**
2. Define MyThread class extending Thread.
3. Inside MyThread constructor:
    - Call super(name) to name the child thread.
    - Call start() to launch the new thread.
4. Override run() method with a loop printing child thread execution steps.
5. In main():
    - Instantiate MyThread (triggers constructor, starts thread automatically).
    - Run a loop printing main thread execution steps with Thread.sleep() delays to clearly observe concurrent interleaving.
6. **Stop**

FLOWCHART



PROGRAM

```
// Custom thread class extending base Thread class
class MyThread extends Thread {

    // Constructor
    public MyThread(String name) {
        // Call base class Thread constructor
```

```
        super(name);
        System.out.println("Child Thread Created: " + getName());
        // Start thread execution (invokes run() asynchronously)
        start();
    }

    // Thread execution entry point
    @Override
    public void run() {
        try {
            for (int i = 1; i <= 5; i++) {
                System.out.println(getName() + " executing step " + i);
                Thread.sleep(500); // Pause for 0.5 sec
            }
        } catch (InterruptedException e) {
            System.out.println(getName() + " interrupted.");
        }
        System.out.println("--> " + getName() + " Finished.");
    }
}

public class Main {
    public static void main(String[] args) {
        // Create and auto-start child thread
        new MyThread("Child-Thread");

        // Concurrent execution in Main Thread
        try {
            for (int i = 1; i <= 5; i++) {
                System.out.println("Main Thread executing step " + i);
                Thread.sleep(500); // Pause for 0.5 sec
            }
        } catch (InterruptedException e) {
            System.out.println("Main Thread interrupted.");
        }
        System.out.println("--> Main Thread Finished.");
    }
}
```

RESULT

EXPERIMENT: 9

AIM
Develop a Java program using an enumeration called Department containing CSE, ISE, ECE, and MECH. Accept a department name as input and display the selected department using valueOf().

THEORY
An **Enumeration (Enum)** in Java is a special data type used to define a collection of fixed constants (e.g., CSE, ISE, ECE, MECH).

- **Type Safety:** Enums restrict input variables to only predefined values, preventing invalid data entry.
- **valueOf() Method:** A built-in static method in all Java enums that converts a string representation of an enum constant into its corresponding enum object. If the string doesn't match any constant, it throws an IllegalArgumentException.

ALGORITHM

1. **Start**
2. Define an enum Department containing constants: CSE, ISE, ECE, MECH.
3. Prompt user to enter a department name as a string.
4. Convert user input to uppercase using .toUpperCase().
5. Enter try block:
    - Call Department.valueOf(input) to obtain the matching enum object.
    - Print "Selected Department: " followed by the enum object.
6. Enter catch block (IllegalArgumentException):
    - Print error message indicating invalid department input.
7. **Stop**

FLOWCHART



PROGRAM

```
import java.util.Scanner;

// Defining Enum outside Main class
enum Department {
```

```
    CSE, ISE, ECE, MECH
}

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter Department Name (CSE, ISE, ECE, MECH): ");
        String input = scanner.nextLine().toUpperCase();

        try {
            // Using valueOf() to map String input to Enum constant
            Department dept = Department.valueOf(input);
            System.out.println("Successfully Selected Department: " + dept);
        } catch (IllegalArgumentException e) {
            System.out.println("Error: Invalid Department Name entered!");
        } finally {
            scanner.close();
        }
    }
}
```

RESULT

EXPERIMENT: 10

AIM

Develop a Java program to read the marks of three subjects as strings, convert them into wrapper objects, calculate the total and average using autoboxing/unboxing and display the result

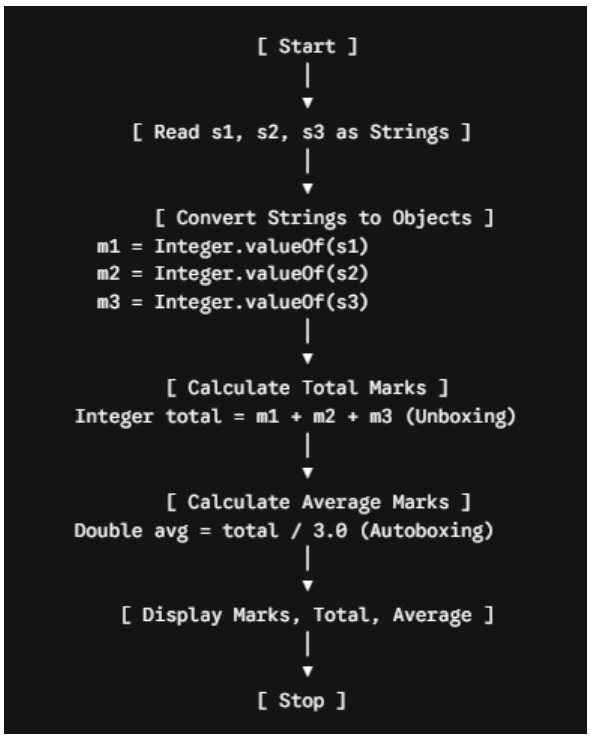
THEORY

- **Wrapper Classes:** Java provides object wrappers (such as Integer and Double) for primitive data types (int, double). Converting a String into an Integer object is done using methods like Integer.valueOf(string).
- **Autoboxing:** Automatic conversion that the Java compiler makes between primitive types and their corresponding object wrapper classes (e.g., converting an int primitive directly to an Integer object).
- **Unboxing:** The reverse process—converting an object of a wrapper class (e.g., Integer) back to its corresponding primitive value (e.g., int) automatically when performing arithmetic operations (+, /).

ALGORITHM

1. **Start**
2. Prompt user to enter three subject marks as String inputs.
3. Parse and convert strings into Integer wrapper objects using Integer.valueOf().
4. Calculate the total score:
    - Perform addition directly on Integer wrapper objects (triggers automatic **unboxing** to primitive int values during arithmetic, then **autoboxes** the result back into an Integer object).
5. Calculate the average score as a Double wrapper object:
    - Unbox total to calculate floating-point division (total / 3.0) and autobox the result into a Double object.
6. Print the individual subject marks, total, and average.
7. **Stop**

FLOWCHART



PROGRAM

```
import java.util.Scanner;

public class Main {
   public static void main(String[] args) {
      Scanner scanner = new Scanner(System.in);
```

```
        // 1. Reading marks as Strings
        System.out.print("Enter marks for Subject 1: ");
        String strMark1 = scanner.nextLine();

        System.out.print("Enter marks for Subject 2: ");
        String strMark2 = scanner.nextLine();

        System.out.print("Enter marks for Subject 3: ");
        String strMark3 = scanner.nextLine();

        // 2. Converting String inputs to Integer Wrapper Objects
        Integer mark1 = Integer.valueOf(strMark1);
        Integer mark2 = Integer.valueOf(strMark2);
        Integer mark3 = Integer.valueOf(strMark3);

        // 3. Calculating Total (Unboxing happens automatically during addition)
        Integer total = mark1 + mark2 + mark3;

        // 4. Calculating Average (Unboxing for division, then Autoboxing to Double)
        Double average = (double) total / 3;

        // 5. Displaying Results
        System.out.println("\n-------------------------------------");
        System.out.println("Subject 1 Marks : " + mark1);
        System.out.println("Subject 2 Marks : " + mark2);
        System.out.println("Subject 3 Marks : " + mark3);
        System.out.println("-------------------------------------");
        System.out.println("Total Marks     : " + total);
        System.out.println("Average Marks   : " + average);
        System.out.println("-------------------------------------");

        scanner.close();
    }
}
```

RESULT

LAB VIVA QUESTIONS:

1. What is the Java Virtual Machine (JVM), and how does it achieve platform independence?
2. Differentiate between JDK, JRE, and JVM.
3. What are Java bytecode and machine code?
4. Explain the execution process of a Java program from source code to runtime.
5. Why is Java not considered a pure object-oriented programming language?
6. What are primitive data types in Java? List all eight.
7. What are reference data types in Java?
8. Why are pointers not explicitly used in Java?
9. What is Just-In-Time (JIT) compiler and what is its role?
10. What is Garbage Collection in Java, and how does System.gc() work?
11. What are class path and environment variables (PATH, CLASSPATH) in Java?
12. Why is the main() method declared as public static void main(String[] args)?
13. What happens if you remove static from the main() method signature?
14. What are Command Line Arguments in Java?
15. What is the difference between System.out.println() and System.out.print()?
16. What are literal values, and what types of literals exist in Java?
17. Explain Type Casting in Java. What is Widening vs Narrowing conversion?
18. What is Automatic Type Promotion in Java expressions?
19. List the operators available in Java and their precedence rules.
20. What is the difference between bitwise & and logical && operators?
21. What is the unsigned right-shift operator (>>>)?
22. Explain the ternary operator ? : with a simple syntax structure.
23. What are control statements in Java? Categorize them.
24. How does a switch statement work, and what data types are allowed in a switch expression?
25. What is the difference between break and continue statements?
26. What are labeled break and continue statements?
27. What is an Object-Oriented Programming System (OOPS)? List its core pillars.
28. Define Class and Object with suitable real-world analogies.
29. What is Encapsulation, and how is it implemented in Java?
30. What is Abstraction, and how does it differ from Encapsulation?
31. What is Inheritance, and what are its types supported in Java?
32. Why does Java not support multiple inheritance with classes?
33. What is Polymorphism? Differentiate between Compile-time and Runtime polymorphism.
34. What is a Constructor? List its characteristics.
35. Differentiate between Default Constructor and Parameterized Constructor.
36. What is Copy Constructor in Java?
37. Can a constructor be declared as private, final, or static?
38. What is Constructor Overloading?
39. What is Constructor Chaining?
40. Explain the usage of the this keyword in Java.
41. How is this() used to invoke a constructor from another constructor?
42. Explain the usage of the super keyword.
43. How do you invoke a superclass constructor using super()?
44. What is Method Overloading? State the rules for method overloading.
45. What is Method Overriding? State the rules for method overriding.
46. Can you override a static method or a private method?
47. What is Dynamic Method Dispatch (Runtime Polymorphism)?
48. What is an Abstract Class? When should you use it?
49. Can an abstract class have constructors, static methods, and instance variables?
50. Can you instantiate an abstract class using the new keyword?
51. What is an Interface in Java?
52. What is the difference between an Abstract Class and an Interface?

53. Can an interface extend another interface in Java?
54. What are Default methods and Static methods in interfaces (Java 8+)?
55. What are Private methods in interfaces (Java 9+)?
56. What is the final keyword? Explain final variables, methods, and classes.
57. What is a Blank Final Variable? How is it initialized?
58. What is the static keyword? Explain static variables, methods, and blocks.
59. What is a Static Initializer Block, and when does it execute?
60. What are Access Modifiers in Java? Explain public, private, protected, and default.
61. What is a Package in Java? What are built-in vs user-defined packages?
62. How do access levels change across packages?
63. What is an Array in Java? How are arrays stored in memory?
64. How do single-dimensional and multidimensional arrays differ in memory allocation?
65. What is ArrayIndexOutOfBoundsException?
66. Why are Strings immutable in Java?
67. What is the String Constant Pool (SCP)?
68. What is the difference between String, StringBuilder, and StringBuffer?
69. What is the difference between == operator and .equals() method for Strings?
70. What is the .intern() method in the String class?
71. What is Exception Handling in Java?
72. Explain the Java Exception Hierarchy (Throwable, Exception, Error).
73. Differentiate between Checked and Unchecked Exceptions.
74. What is the difference between Error and Exception?
75. Explain the role of try, catch, and finally blocks.
76. Can a try block exist without a catch block?
77. Can we write multiple catch blocks for a single try block? What are the ordering rules?
78. What is the difference between throw and throws keywords?
79. How do you create a User-Defined (Custom) Exception in Java?
80. What is Multithreading, and what are its advantages?
81. What are the two ways to create a thread in Java?
82. Differentiate between extending Thread class and implementing Runnable interface.
83. What is the Life Cycle of a Thread? List all thread states.
84. Differentiate between .start() and .run() methods in multithreading.
85. What is the purpose of Thread.sleep(), Thread.yield(), and Thread.join()?
86. What is Thread Synchronization, and why is it necessary?
87. What is a synchronized block vs a synchronized method?
88. What is Deadlock in multithreading, and how can it be avoided?
89. What is Inter-Thread Communication? Explain wait(), notify(), and notifyAll().
90. Why are wait(), notify(), and notifyAll() defined in the Object class rather than Thread class?
91. What is an Enumeration (enum) in Java?
92. How does valueOf() and values() work in Java Enums?
93. What are Wrapper Classes in Java? Name all primitive wrapper types.
94. What is Autoboxing and Unboxing in Java?
95. What is the difference between Integer.parseInt() and Integer.valueOf()?
96. What is the Object class in Java? List some of its key methods.
97. What is the .toString() method, and why should it be overridden?
98. What is the .hashCode() and .equals() contract in Java?
99. What is garbage collection eligibility, and what was the finalize() method?
100. What is the Java Collections Framework? Name its core interfaces (Collection, List, Set, Map).